

WAREHOUSE MANAGEMENT SYSTEM

Group ID: 01

Dương Anh Hà

Huỳnh Phúc Nguyên

Lâm Hồng Hải Hoàng Lê

Nguyễn Việt Bách

Đỗ Minh Khang

25127045

25127106

25127207

25127275

25127363



Table of contents

01

Overview

02

**Domain
Layer**

03

**Repository
Layer**

04

**Service
Layer**

05

**GUI
Layer**

06

**Design
Patterns**

07

Limitations

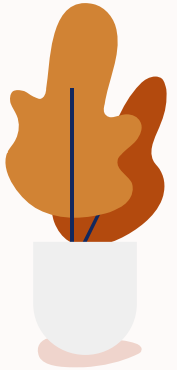
08

**Future
Enhancements**

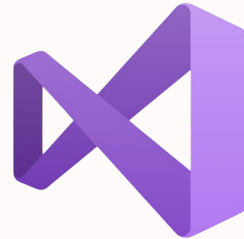


01

Overview



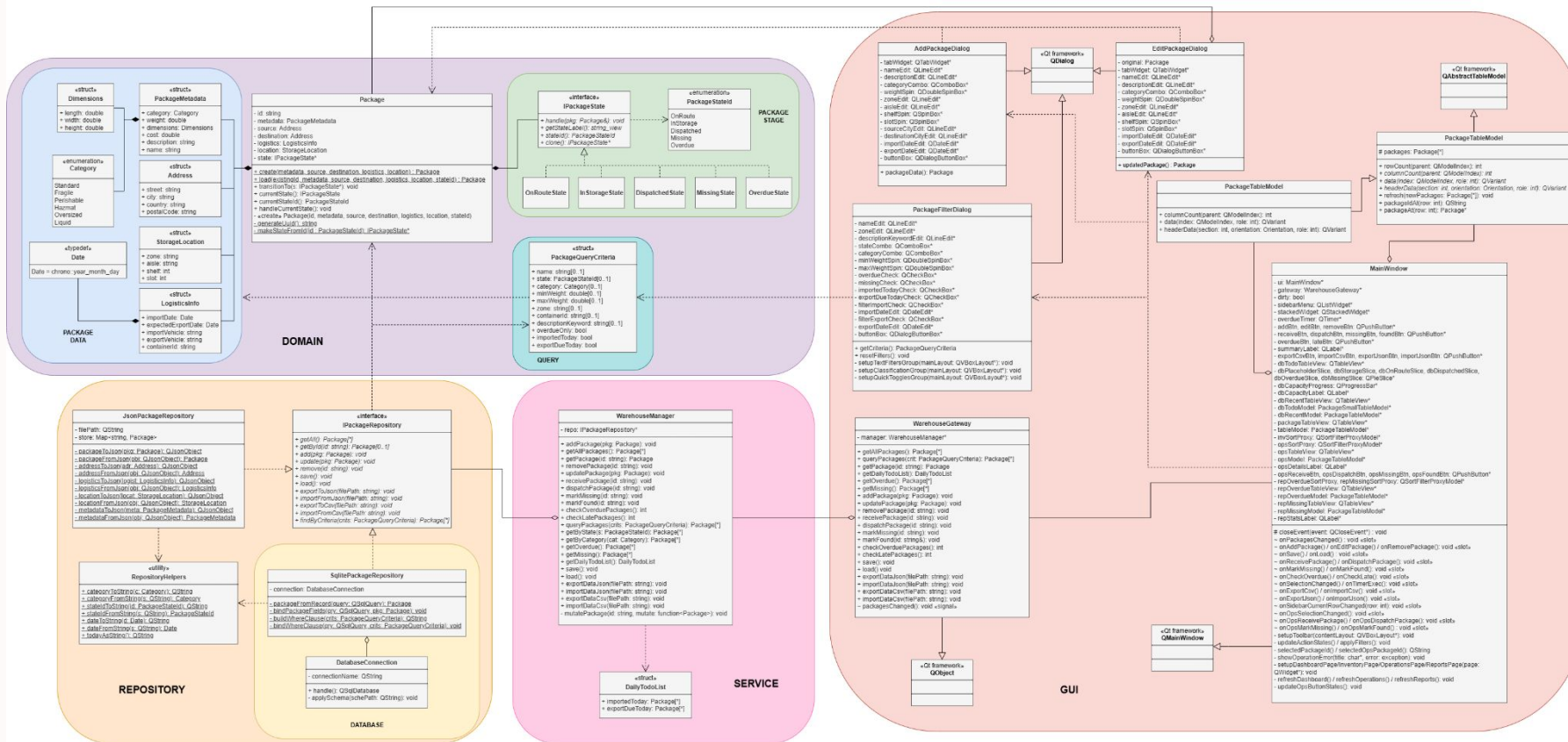
TOOLS



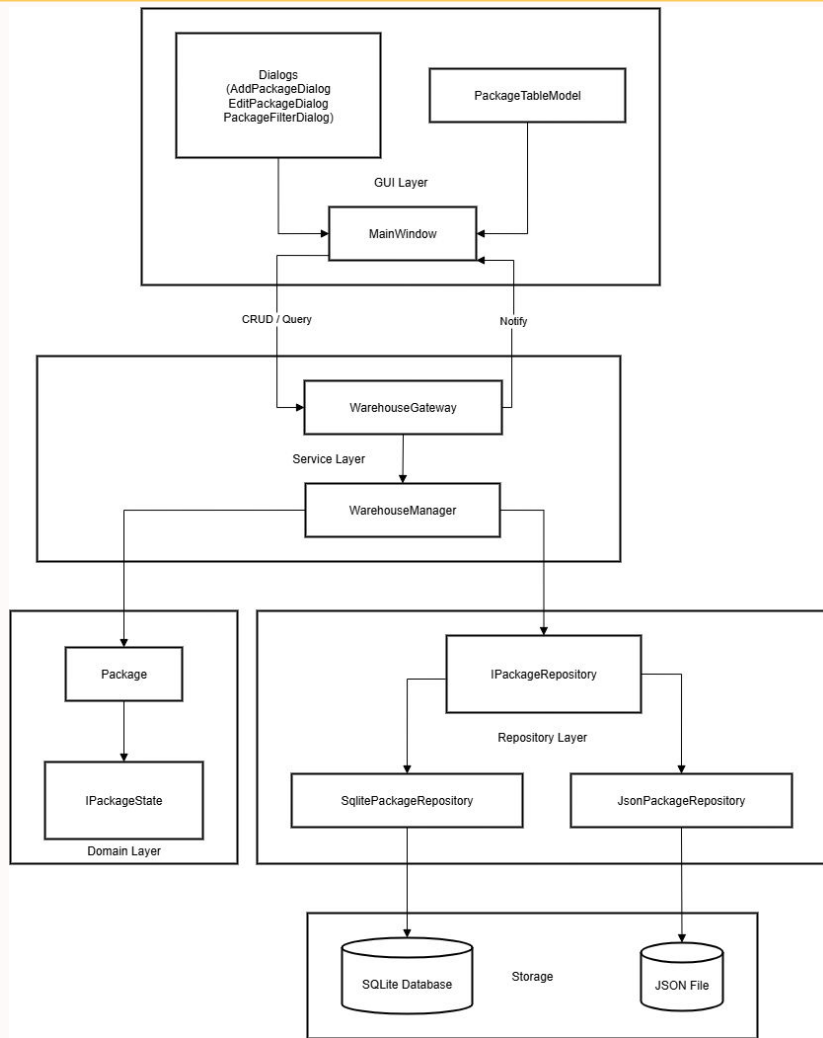
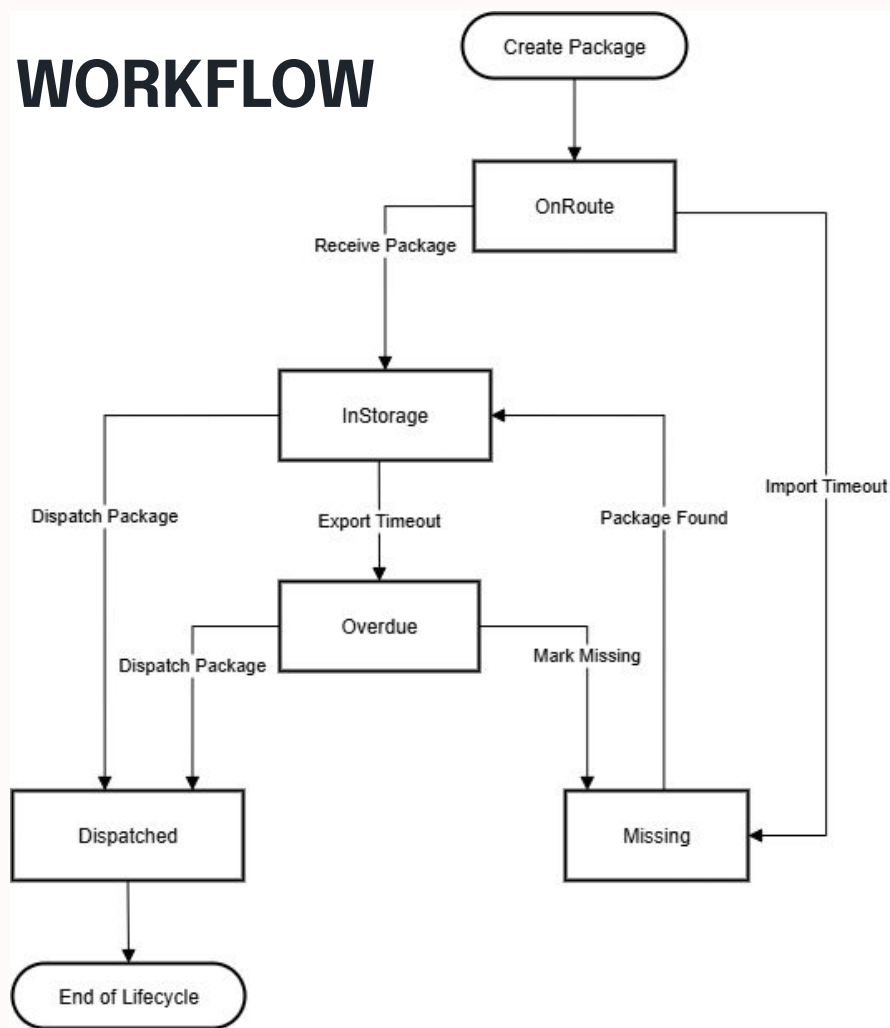
PROJECT ARCHITECTURE

LAYERS
GUI
Service
Repository
Domain

UML CLASSES DIAGRAM



WORKFLOW



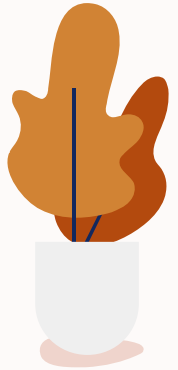
CORE CHARACTERISTICS

- Package Registration
- Package Tracking
- Inventory Monitoring
- Package Search
- Warehouse Exception Handling



02

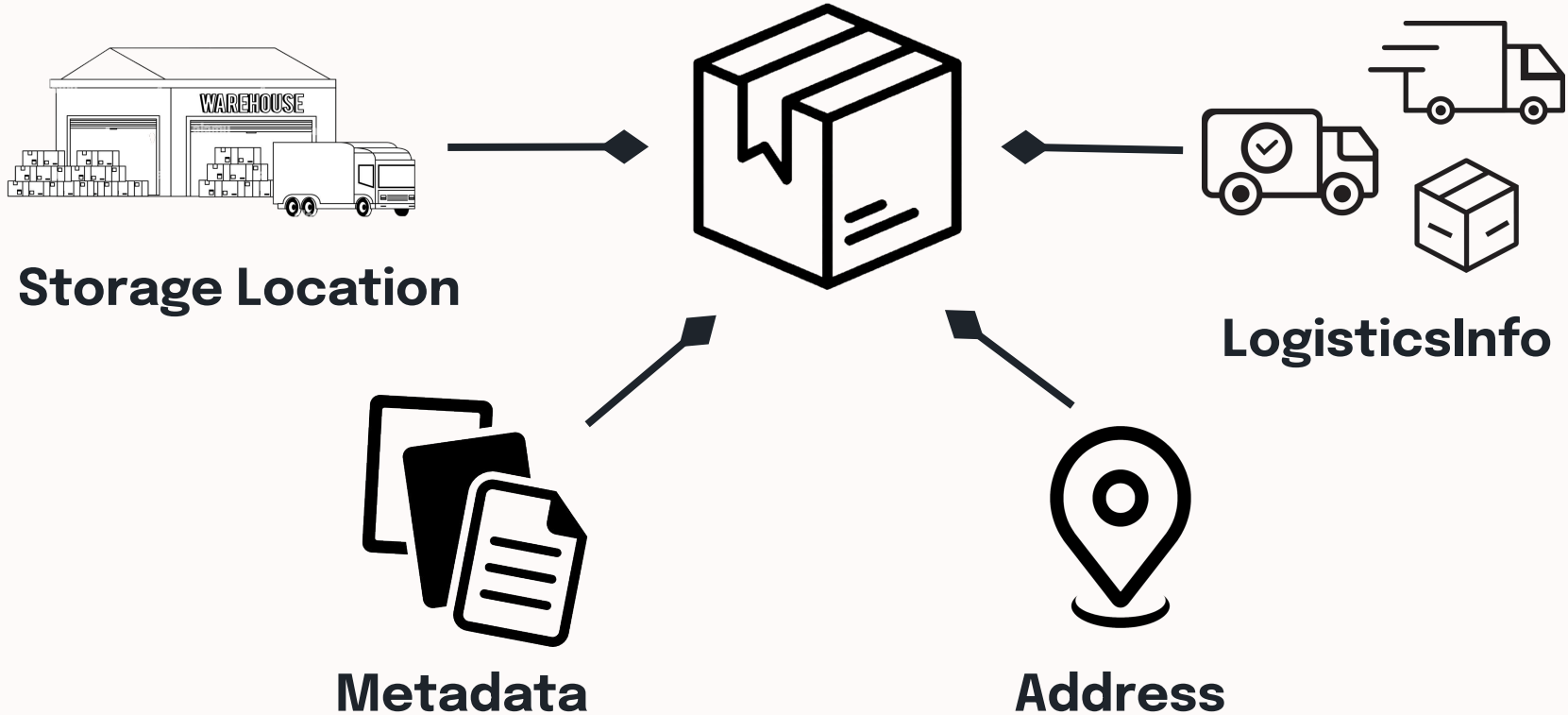
Domain Layer



Characteristics

- Contains the system core business logic:
- + Package data
- + Package stage
- + Query
- Defines entities and business constraints.
- Stays independent of UI and infrastructure.

Package data

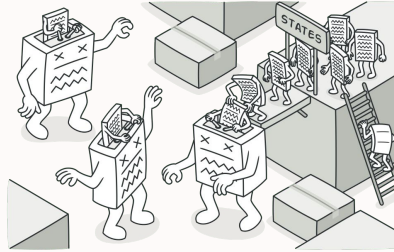


Package Stage

0. Onroute



IPackageState



1. Instorage



2. Dispatched



3. Missing



4. Overdue



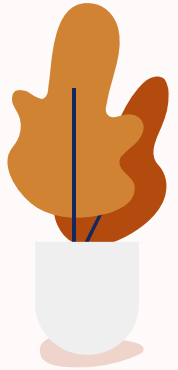
Package Query Criteria

- Acts as a query filter object
- Encapsulates search criteria in one place
- Supports separation of concerns (domain defines *what to filter* and repository/application handles *how to fetch*.)



03

Repository Layer



Characteristics

- Responsible for data persistence in the system
- Stores and retrieves data
- Supports two data storage methods:
 - + Json files
 - + SQLite database
- Keeps the service layer independent of storage details

IPackageRepository

- Abstract interface for all package repositories
- Defines CRUD, import/export, and filtering methods
- Allows the system to switch storage backends easily

Data storage comparison

Aspect	JsonPackageRepository	SqlitePackageRepository
Data Storage	Stores data in a JSON file	Stores data in a SQLite database
Data structure	All packages are stored as a JSON array	Each package is stored as a row in the packages table
Read/Write Method	Loads the entire JSON file into memory before processing	Executes queries directly on the database
Data management	Uses an unordered_map as an in-memory cache	Does not use a primary cache; interacts directly with the database
Search	findByCriteria() filters data in memory	findByCriteria() generates SQL queries and filters data in the database

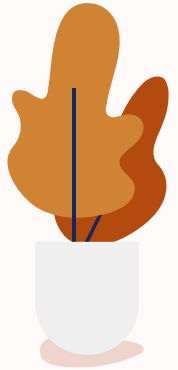
Supporting Files

- DatabaseConnection
 - + Opens and manages the SQLite connection
 - + Loads the database schema
- RepositoryHelpers
 - + Converts enums and dates to string formats
 - + Shared by both JSON and SQLite repositories



04

Service Layer



Warehouse Manager

- Handles package lifecycle actions
- Enforces valid state changes
- Provides daily operational check
- Uses criteria-based queries for cleaner and faster filtering
- Delegates persistence to repository

Package workflow coordinator

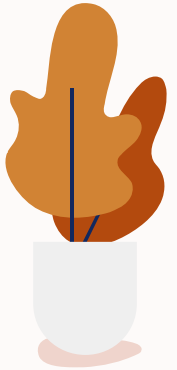
UI/Controller → WarehouseManager → IPackageRepository → Data

WarehouseManager ↔ Domain (Package, State, Criteria)



05

GUI Layer



Characteristics

- Connects user actions to backend service
- Focus on usability and real-time data view
- Workflow:

User action → Gateway → Signal → UI refresh

ARCHITECTURE

- Mainwindow: UI orchestration
- PackageTableModel: Data adapter
- WarehouseGateway: GUI-service bridge
- Dialogs: Collecting user input for specific tasks

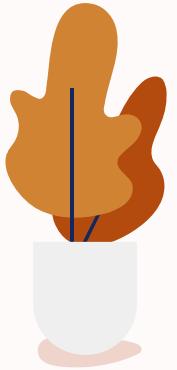
Main features

- 4 pages: Dashboard / Inventory / Operations / Reports
- CRUD + state transitions
- Sorting/filtering + status highlights
- Import/Export CSV and JSON



06

Design Patterns



Design Patterns

Pattern	Location	Benefits
State	IPackageState and its concrete state classes	Separates behavior by state, improving maintainability and extensibility.
Observer	WarehouseGateway and QxSignal-slot mechanism	Keeps components synchronized while reducing coupling.
Repository	IPackageRepository abstraction; JsonPackageRepository; SqlitePackageRepository	Separates data access from business logic, improving maintainability.

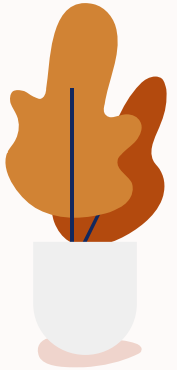
Design Patterns

Pattern	Location	Benefits
Dependency Injection	WarehouseManager receives IPackageRepository via its constructor; MainWindow injects WarehouseManager	Increases flexibility, testability, and component reusability.
Query Object	PackageQueryCriteria, translated by SqlitePackageRepository::buildWhereClause()	Encapsulates search criteria for easier extension and optimization.



07

Limitations



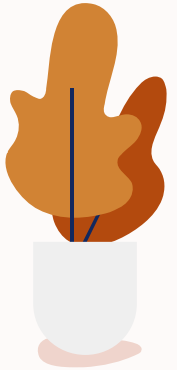
Limitations

- Lack of Automated Testing:
- Incomplete Database Constraints:
- No User Authentication and Authorization:
- Limited JSON Repository Performance:



08

Future enhancements



Future Enhancements

- Implement Container and Employee modules.
- Add foreign key constraints for improved data integrity.
- Introduce comprehensive unit testing.
- Implement user authentication and role-based access control.
- Optimize JSON repository performance with indexing.
- Continue using SQLite as the preferred backend for large-scale deployments.

THANK YOU !!!